RETAILMART V3 · SQL

Views & Materialized Views

— Siraj Sir

📱 WhatsApp Announcement

Views & Materialized Views

Views are how analysts share tested, reusable logic with their team. Materialized Views are how analysts make expensive queries fast. Today you learn both -- the analyst's delivery format.

Part 1: CREATE VIEW -- Reusable Query Logic
Part 2: Layered Views -- Raw to Cleaned to Metrics
Part 3: Materialized Views -- Precomputed Results

By the end of today, you will build a 3-layer view stack that powers a dashboard: raw data -> cleaned data -> monthly KPIs. This is exactly what the capstone project requires.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: CREATE VIEW	2 Hours	What a view is and why analysts use them, CREATE VIEW syntax, Querying views like tables
Part 2: Layered Views		Raw -> cleaned -> enriched -> metrics, View dependencies and DROP CASCADE, When to layer vs keep flat
Part 3: Materialized Views		CREATE MATERIALIZED VIEW, REFRESH and CONCURRENTLY, View vs MV decision: freshness vs speed

YOUR SQL JOURNEY

- ✓ Query Plans
- ✓ Indexing
- ✓ Pivoting & FILTER
- ✓ Median & DISTINCT ON
- ✓ Date/Time Mastery
- ✓ JSON & JSONB
- 25 **Views & MVs** ← HERE
- 26 Data Quality

RECAP

What We Covered Yesterday

- JSONB operators: -> (JSONB), ->> (TEXT), @> (containment)
- jsonb_array_elements for unnesting JSON arrays
- Flattening JSONB into typed columns for analysis

STORY

The Query You Write 50 Times

You write the same 4-table join with the same WHERE filters every week for the sales report. Your colleague writes it slightly differently and gets different numbers. Views solve both problems: write the logic once, name it, and everyone queries the same thing.

DEFINITION

View = Saved Query with a Name

- A view is a named SELECT statement stored in the database
- It does NOT store data -- it re-executes the query each time
- Querying a view is identical to querying a table
- Changes to underlying tables are immediately reflected

View Commands

-- Create View

```
CREATE VIEW schema.view_name AS
SELECT ... FROM ... WHERE ...;
```

-- Replace View

```
CREATE OR REPLACE VIEW schema.v_name AS
SELECT ...;  -- updates definition
```

-- Drop View

```
DROP VIEW schema.view_name;
DROP VIEW IF EXISTS schema.v_name CASCADE;
```

Simple View (1/2)

```
CREATE VIEW analytics.v_orders_clean AS
SELECT
  o.order_id,
  o.cust_id,
  o.store_id,
  o.order_date,
  o.net_total,
  o.order_status,
  c.first_name || ' ' || c.last_name AS customer_name,
  c.email,
  s.store_name,
  dr.region_name AS region
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
JOIN stores.stores s
  ON o.store_id = s.store_id
JOIN core.dim_region dr
  ON s.region_id = dr.region_id;
```

Simple View (2/2)

```
-- Now anyone can query:
SELECT region, COUNT(*), SUM(net_total)
FROM analytics.v_orders_clean
WHERE order_status = 'Delivered'
GROUP BY region;
```

PRO TIP

PRO TIP

Views are not faster than the underlying query -- they just save you from rewriting it. For speed, use materialized views (Part 3). For reusability and consistency, use regular views.

STORY

The Three-Layer Cake

Professional analytics teams build views in layers. Layer 1 cleans and joins raw data. Layer 2 enriches with business logic. Layer 3 aggregates into KPIs. Each layer is simple. Together, they produce a complex result. This is the pattern for the capstone.

Layer 1 View

```
CREATE VIEW analytics.v_orders_clean AS
SELECT
  o.order_id,
  o.cust_id,
  o.order_date,
  o.net_total,
  o.order_status,
  s.store_name,
  dr.region_name AS region,
  pm.mode_name AS payment_mode
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
JOIN finance.payment_modes pm
  ON o.payment_mode_id = pm.mode_id;
```

Layer 2 View

```
CREATE VIEW analytics.v_orders_enriched AS
SELECT
  *,
  DATE_TRUNC('month', order_date)::DATE AS order_month,
  EXTRACT(DOW FROM order_date)::INT      AS day_of_week,
  CASE
    WHEN net_total >= 10000 THEN 'high_value'
    WHEN net_total >= 2000  THEN 'medium_value'
    ELSE 'low_value'
  END                                   AS value_segment
FROM analytics.v_orders_clean;
```

Layer 3 View

```
CREATE VIEW analytics.v_monthly_kpis AS
SELECT
  region,
  order_month,
  COUNT(*)                              AS total_orders,
  COUNT(*) FILTER (WHERE order_status = 'Delivered')
                                        AS delivered,
  SUM(net_total)                     AS revenue,
  ROUND(AVG(net_total)::NUMERIC, 2)
                                        AS avg_order_value,
  COUNT(DISTINCT cust_id)             AS unique_customers
FROM analytics.v_orders_enriched
GROUP BY region, order_month;
```

View Dependencies

v_monthly_kpis depends on v_orders_enriched, which depends on v_orders_clean. Dropping v_orders_clean without CASCADE will fail:

Drop Cascade

```
-- This FAILS (has dependents):
DROP VIEW analytics.v_orders_clean;

-- This works (drops all dependents):
DROP VIEW analytics.v_orders_clean CASCADE;
-- WARNING: also drops v_orders_enriched and v_monthly_kpis!

-- Check dependencies before dropping:
SELECT dependent_ns.nspname || '.' || dependent_v.relname
  AS dependent_view
FROM pg_depend
JOIN pg_rewrite ON pg_depend.objid = pg_rewrite.oid
JOIN pg_class AS dependent_v
  ON pg_rewrite.ev_class = dependent_v.oid
JOIN pg_namespace AS dependent_ns
  ON dependent_v.relnamespace = dependent_ns.oid
WHERE pg_depend.refobjid = 'analytics.v_orders_clean'::REGCLASS;
```

Practice 1: Create a Basic View

EASY

SCENARIO: Analysts repeatedly join orders with customers and stores.

TASK: Create a view called analytics.v_order_details that joins orders, customers, and stores. Include: order_id, order_date, net_total, order_status, customer name, email, store_name, region.

Answer

✅ ANSWER

```
CREATE VIEW analytics.v_order_details AS
SELECT
  o.order_id,
  o.order_date,
  o.net_total,
  o.order_status,
  c.first_name || ' ' || c.last_name
    AS customer_name,
  c.email,
  s.store_name,
  dr.region_name AS region
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
JOIN stores.stores s
  ON o.store_id = s.store_id
JOIN core.dim_region dr
  ON s.region_id = dr.region_id;
```

Practice 2: Layered View with Cleaning Rules

MEDIUM

SCENARIO: Build a Layer 2 view that applies cleaning rules on top of Practice 1's view.

TASK: Create analytics.v_order_details_enriched that adds: order_month, value_segment (high/medium/low based on net_total), and is_weekend (boolean).

Answer

✅ ANSWER

```
CREATE VIEW analytics.v_order_details_enriched AS
SELECT
  *,
  DATE_TRUNC('month', order_date)::DATE
    AS order_month,
  CASE
    WHEN net_total >= 10000 THEN 'high_value'
    WHEN net_total >= 2000  THEN 'medium_value'
    ELSE 'low_value'
  END AS value_segment,
  EXTRACT(DOW FROM order_date) IN (0, 6)
    AS is_weekend
FROM analytics.v_order_details;
```

STORY

The 30-Second Dashboard

Your monthly KPI view takes 30 seconds because it aggregates 350,000 order items. The dashboard times out. A Materialized View precomputes and stores the result. The dashboard reads cached data in 50ms. You refresh it once per hour (or daily) to keep it current.

DEFINITION

Key Differences

View: Saves the QUERY. Re-executes every time you query it. Always shows current data. Can be slow for complex queries.

Materialized View (MV): Saves the RESULT. Returns precomputed data instantly. Must be manually refreshed to reflect changes. Fast reads, stale data between refreshes.

Materialized View Syntax

```
CREATE MATERIALIZED VIEW analytics.mv_monthly_revenue AS
SELECT
  dr.region_name AS region,
  DATE_TRUNC('month', o.order_date)::DATE AS month,
  COUNT(*) AS orders,
  SUM(o.net_total) AS revenue,
  ROUND(AVG(o.net_total)::NUMERIC, 2) AS avg_value
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name, DATE_TRUNC('month', o.order_date);

-- Query it like a table (instant!):
SELECT * FROM analytics.mv_monthly_revenue
WHERE region = 'North'
ORDER BY month;
```

Refresh Commands

```
-- Full refresh (locks the MV during refresh):
REFRESH MATERIALIZED VIEW analytics.mv_monthly_revenue;

-- Concurrent refresh (no lock, needs unique index):
CREATE UNIQUE INDEX ON analytics.mv_monthly_revenue
  (region, month);

REFRESH MATERIALIZED VIEW CONCURRENTLY
  analytics.mv_monthly_revenue;

-- CONCURRENTLY allows reads during refresh.
-- Requires a unique index on the MV.
```

When to Use View vs MV

- Use a VIEW when: data must be real-time, query is fast, or the view is just for code reuse
- Use an MV when: query is slow (> 5 seconds), data can be slightly stale, dashboard needs instant response
- Refresh strategy: daily for batch reports, hourly for near-real-time dashboards

PRO TIP

PRO TIP

For REFRESH CONCURRENTLY, the MV must have a UNIQUE INDEX. Without it, CONCURRENTLY fails. The unique index must cover columns that uniquely identify each row in the MV result.

Practice 3: MV with Concurrent Refresh

HARD

SCENARIO: The dashboard needs a fast monthly revenue summary. Create an MV with concurrent refresh support.

TASK: Create an MV called analytics.mv_revenue_summary with: region, month, orders, revenue, avg_order_value. Add the unique index needed for CONCURRENTLY. Then refresh it.

Answer

✅ ANSWER

```
CREATE MATERIALIZED VIEW analytics.mv_revenue_summary AS
SELECT
  dr.region_name AS region,
  DATE_TRUNC('month', o.order_date)::DATE AS month,
  COUNT(*) AS orders,
  SUM(o.net_total) AS revenue,
  ROUND(AVG(o.net_total)::NUMERIC, 2)
    AS avg_order_value
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
WHERE o.order_status = 'Delivered'
GROUP BY dr.region_name,
  DATE_TRUNC('month', o.order_date);

-- Unique index for CONCURRENTLY:
CREATE UNIQUE INDEX ON analytics.mv_revenue_summary
  (region, month);

-- Concurrent refresh (no downtime):
REFRESH MATERIALIZED VIEW CONCURRENTLY
  analytics.mv_revenue_summary;
```

Practice 4: 3-Layer View Stack (Lab)

CRAZY

SCENARIO: Build the complete view stack for the capstone project pattern.

TASK: Create all three layers: (1) v_orders_clean (join orders + customers + stores + payment_modes), (2) v_orders_enriched (add month, value_segment, day_of_week), (3) mv_monthly_kpis (MV: region, month, orders, revenue, unique_customers, cancel_rate with concurrent refresh support).

Answer (1/2)

✅ ANSWER

```
-- Layer 1: Clean
CREATE VIEW analytics.v_orders_clean AS
SELECT o.order_id, o.cust_id,
  o.order_date, o.net_total, o.order_status,
  s.store_name, dr.region_name AS region,
  pm.mode_name AS payment_mode
FROM sales.orders o
JOIN stores.stores s ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
JOIN finance.payment_modes pm
  ON o.payment_mode_id = pm.mode_id;

-- Layer 2: Enrich
CREATE VIEW analytics.v_orders_enriched AS
SELECT *,
  DATE_TRUNC('month', order_date)::DATE AS order_month,
  CASE WHEN net_total >= 10000 THEN 'high'
       WHEN net_total >= 2000 THEN 'medium'
       ELSE 'low' END AS value_segment,
  EXTRACT(DOW FROM order_date)::INT AS dow
FROM analytics.v_orders_clean;
```

Answer (2/2)

✅ ANSWER

```
-- Layer 3: MV KPIs
CREATE MATERIALIZED VIEW analytics.mv_monthly_kpis AS
SELECT region, order_month,
  COUNT(*) AS orders,
  SUM(net_total) AS revenue,
  COUNT(DISTINCT cust_id) AS customers,
  ROUND(COUNT(*) FILTER
    (WHERE order_status = 'Cancelled')
    * 100.0 / NULLIF(COUNT(*), 0), 1)
    AS cancel_rate
FROM analytics.v_orders_enriched
GROUP BY region, order_month;

CREATE UNIQUE INDEX ON analytics.mv_monthly_kpis
  (region, order_month);
```

INTERVIEW

Q: What is the difference between a View and a Materialized View?

A: A View stores only the query definition -- it re-executes the SELECT every time you query it, always returning current data. A Materialized View stores the query RESULT on disk -- it returns precomputed data instantly but must be manually refreshed to reflect changes. Use Views for real-time needs and code reuse. Use MVs for expensive queries powering dashboards.

INTERVIEW

Q: What is REFRESH MATERIALIZED VIEW CONCURRENTLY?

A: CONCURRENTLY refreshes the MV without locking it -- users can still read the old data while the refresh runs. Without CONCURRENTLY, the MV is locked during refresh and queries block. CONCURRENTLY requires a UNIQUE INDEX on the MV to identify which rows changed.

HW 1: Create a Product Analysis View

EASY

TASK: Create a view analytics.v_product_sales joining products, order_items, orders, dim_brand, dim_category. Include: product_id, product_name, category_name, order_count, total_quantity_sold, total_revenue.

Answer

✅ ANSWER

```
CREATE VIEW analytics.v_product_sales AS
SELECT
  p.product_id,
  p.product_name,
  cat.category_name,
  COUNT(DISTINCT oi.order_id) AS order_count,
  SUM(oi.quantity)            AS total_qty_sold,
  SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM products.products p
JOIN core.dim_brand b ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
LEFT JOIN sales.order_items oi
  ON p.product_id = oi.prod_id
LEFT JOIN sales.orders o
  ON oi.order_id = o.order_id
  AND o.order_status = 'Delivered'
GROUP BY p.product_id, p.product_name,
  cat.category_name;
```

HW 2: MV for Slow Category Report

MEDIUM

TASK: Convert a slow category-level monthly report into an MV. Include: category_name, month, revenue, order_count, avg_price. Add the unique index for concurrent refresh.

Answer

✅ ANSWER

```
CREATE MATERIALIZED VIEW
  analytics.mv_category_monthly AS
SELECT
  cat.category_name,
  DATE_TRUNC('month', o.order_date)::DATE AS month,
  SUM(oi.quantity * oi.unit_price)  AS revenue,
  COUNT(DISTINCT o.order_id)        AS order_count,
  ROUND(AVG(oi.unit_price)::NUMERIC, 2)
                                    AS avg_price
FROM sales.order_items oi
JOIN sales.orders o ON oi.order_id = o.order_id
JOIN products.products p ON oi.prod_id = p.product_id
JOIN core.dim_brand b ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
WHERE o.order_status = 'Delivered'
GROUP BY cat.category_name,
  DATE_TRUNC('month', o.order_date);

CREATE UNIQUE INDEX ON analytics.mv_category_monthly
  (category_name, month);
```

HW 3: View Dependency Check

MEDIUM

TASK: Write a query that lists all views and MVs in the analytics schema. Show the view name, type (view vs materialized view), and definition.

Answer

✅ ANSWER

```
SELECT
  c.relname AS view_name,
  CASE c.relkind
    WHEN 'v' THEN 'view'
    WHEN 'm' THEN 'materialized_view'
  END AS type,
  pg_get_viewdef(c.oid, true) AS definition
FROM pg_class c
JOIN pg_namespace n ON c.relnamespace = n.oid
WHERE n.nspname = 'analytics'
  AND c.relkind IN ('v', 'm')
ORDER BY c.relkind, c.relname;
```

HW 4: Refresh Strategy Design

HARD

TASK: You have three MVs: (1) mv_hourly_orders (refreshed every 15 minutes), (2) mv_daily_revenue (refreshed once per day), (3) mv_monthly_kpis (refreshed once per week). Write the REFRESH commands for each and a shell script that runs them in order.

Answer

✅ ANSWER

```
-- In PostgreSQL:
REFRESH MATERIALIZED VIEW CONCURRENTLY
  analytics.mv_hourly_orders;
REFRESH MATERIALIZED VIEW CONCURRENTLY
  analytics.mv_daily_revenue;
REFRESH MATERIALIZED VIEW CONCURRENTLY
  analytics.mv_monthly_kpis;

-- Shell script (refresh_all.sh):
-- #!/bin/bash
-- psql -U postgres -d accio_retailmart_ -c \
--   "REFRESH MATERIALIZED VIEW CONCURRENTLY
--    analytics.mv_hourly_orders;"
-- psql -U postgres -d accio_retailmart_ -c \
--   "REFRESH MATERIALIZED VIEW CONCURRENTLY
--    analytics.mv_daily_revenue;"
-- psql -U postgres -d accio_retailmart_ -c \
--   "REFRESH MATERIALIZED VIEW CONCURRENTLY
--    analytics.mv_monthly_kpis;"
-- echo 'All MVs refreshed.'
```

HW 5: Track Dependencies Before Drop

CRAZY

TASK: Before dropping analytics.v_orders_clean, list all views and MVs that depend on it. Write the dependency query, then show the safe drop order (dependents first, base last).

Answer (1/2)

✅ ANSWER

```
-- List dependents:
WITH RECURSIVE deps AS (
  SELECT DISTINCT
    dependent_v.relname AS view_name,
    dependent_v.relkind AS kind,
    1 AS depth
  FROM pg_depend d
  JOIN pg_rewrite rw ON d.objid = rw.oid
  JOIN pg_class dependent_v
    ON rw.ev_class = dependent_v.oid
  WHERE d.refobjid =
    'analytics.v_orders_clean'::REGCLASS
    AND dependent_v.relname != 'v_orders_clean'
)
SELECT view_name,
  CASE kind WHEN 'v' THEN 'view'
    WHEN 'm' THEN 'materialized_view' END AS type
FROM deps ORDER BY depth DESC;

-- Safe drop order (dependents first):
-- 1. DROP MATERIALIZED VIEW analytics.mv_monthly_kpis;
-- 2. DROP VIEW analytics.v_orders_enriched;
```

Answer (2/2)

✅ ANSWER

```
-- 3. DROP VIEW analytics.v_orders_clean;
```

KEY TAKEAWAYS

Views save query logic: CREATE VIEW names a SELECT statement. Everyone queries the same tested logic.

Layer views for clarity: Raw -> cleaned -> enriched -> metrics. Each layer is simple.

MVs store precomputed results: CREATE MATERIALIZED VIEW caches the result on disk. Instant reads, manual refresh.

CONCURRENTLY needs a unique index: Without it, REFRESH CONCURRENTLY fails. Plan the unique key when creating the MV.

CASCADE is dangerous: DROP VIEW CASCADE deletes all dependent views. Always check dependencies first.

What is Next

Tomorrow we tackle Data Quality, Deduplication & Cleansing -- finding bad data, deduping with ROW_NUMBER, standardizing with regex, and building data quality checks. This is the last day of new concepts before the capstone weeks.